Role Assignment

Member	Role	Covers
Ansh	Frontend + Problem	intro, what was built, core problem, live UI demo
Apoorva	Backend + Architecture	tech stack, system architecture, data structure evolution
Sameer	Go Engine + Results	Go integration, matching algorithm, benchmarks, takeaways

---

Full Scripts (Updated)

SEGMENT 0 — Title Card `[0:00 – 0:10]`

```
SCREEN SHOWS (static, 10 seconds):

  STOCK ORDER BOOK
  Real-Time Matching Engine

  Apoorva   S24BCAU0120
  Ansh      S24BCAU0102
  Sameer    S24BCAU0106

  Bennett University
  School of Computer Science Engineering and Technology
  DTI Course — May 2026
  Mentored by: Dr. Pallav Bishi
```

SEGMENT 1 — Team Intro `[0:10 – 0:35]`

```
ANSH (on camera):
"Hi, I'm Ansh. I handled the frontend side of the project —
the React interface, real-time order book display, and user testing."

APOORVA (on camera):
"I'm Apoorva. I worked on the backend architecture, the API layer,
and the different versions of the order book data structure."
```

```
SAMEER (on camera):
"And I'm Sameer. I built the Go matching engine integration
and handled the performance benchmarking side."
```

SEGMENT 2 — Ansh: Problem + Live Demo [0:35 – 1:45]

Format: Face-on-camera for problem statement, then switch to screen recording of the running app.

```
ANSH (on camera — ~25 seconds):

"So, platforms like Paathshaala and Moneybhai already let
students do simulated trading. But they never show you how
a trade actually executes under the hood — the matching engine,
the order book, the data structures.

That's the gap we tried to fill. We built an open-source
simulator where the technical side is fully visible —
from the UI all the way down to the matching logic."

--- SWITCH TO SCREEN RECORDING ---

ANSH (voiceover while showing live app, ~40 seconds):

"This is the running application. Three panels —
order book on the left showing live bids and asks,
order placement in the center, and trade history on the right.

[SHOW: placing a buy order on screen]

I'm placing a buy order here — watch the portfolio balance
change after it matches.

[SHOW: balance updating]

That update happens through a WebSocket connection,
so all connected users see it instantly.
The whole thing runs with just a git clone and npm run dev —
no paid services, no external dependencies."
```

SEGMENT 3 — Apoorva: Architecture + Data Structures [1:45 – 3:00]

Format: Primarily face-on-camera. Can optionally show VS Code or a diagram briefly.

```
APOORVA (on camera — architecture, ~30 seconds):

"The system has three layers. React frontend, Node.js server,
and a Go matching engine running as a separate service.

The frontend sends orders through tRPC to Node.js.
Node.js verifies auth using Firebase, checks portfolio balance,
then sends the order to the Go service over HTTP.
Go processes it, returns the result, and Node.js
broadcasts the update via Socket.io to every connected client.

The key thing is — the React frontend has no idea
whether matching is done in TypeScript or Go.
That separation is what let us upgrade the engine
without touching the UI at all."

--- APOORVA continues (data structures, ~35 seconds) ---

"Now, the most interesting part technically was the
order book data structure. We actually built three versions.

The first, naive version, pushed all orders into an array
and re-sorted the whole thing on every insert.
That's O(n log n) on every add. It works, but it doesn't scale.

The second, baseline version used HashMaps for price-level
lookups and sorted arrays for best bid and ask access.
Much better for reads, but order removal still needed
array splicing — still O(n) on cancel.

The third, optimized version uses a doubly-linked list
at each price level. Cancelling an order becomes
a pointer swap — O(1), no shifting.
Benchmarks showed roughly a 6x difference between
the worst and best version."
```

SEGMENT 4 — Sameer: Go Engine + Benchmarks [3:00 – 4:15]

Format: Face-on-camera for explanation, then switch to terminal screen recording.

SAMEER (on camera — ~25 seconds):

"After the TypeScript order book was optimized,
we went one step further. Node.js runs on V8 —
a garbage-collected JavaScript runtime.
For a compute-heavy loop like matching thousands of orders,
that GC overhead adds up.

Go compiles to a native binary. It's thread-safe by design,
uses roughly half the memory, and has predictable latency
without GC pauses.

So we built the matching engine in Go as a separate microservice,
wired it into the existing Node.js backend,
and ran it side by side with the TypeScript version."

--- SWITCH TO TERMINAL SCREEN RECORDING ---

SAMEER (voiceover while showing benchmark terminal output, ~40 seconds):

"This is the actual benchmark output.

[SHOW: TypeScript benchmark running]

TypeScript hits around 2.4 million orders per second
at 1k orders — but watch what happens at 10k.

[HIGHLIGHT: 10k row — 192,436 orders/sec]

It drops to 192k. That's a V8 garbage collection event
kicking in. Not a code bug — it's the runtime.

[SHOW: Go benchmark running]

Go at 10k orders — 3.6 million per second,
consistent all the way through.
Single order latency drops from around 100 microseconds

in TypeScript to about 5 microseconds in Go.

And because the matching engine was already isolated
as its own module, the integration was mostly just
adding an HTTP client on the Node.js side."

SEGMENT 5 — All Three: Takeaways + Close [4:15 – 5:00]

ANSH:
"The biggest takeaway for me was that real user testing
finds bugs that code review never does. We had a silent
validation failure on decimal quantities — found it only
when someone actually tried to use the form."

APOORVA:
"For me it was the data structure work. The difference
between O(n) and O(1) on removal is not just theory —
you see it in the benchmark numbers."

SAMEER:
"And separating the matching engine from the start
is what made the Go migration possible without breaking anything.
Clean architecture is not just a good practice,
it's what makes improvements actually doable."

[ALL THREE, brief]:
"The full code and documentation are open source —
link is in the description."

SEGMENT 6 — Closing Card [4:50 – 5:00]

SCREEN SHOWS (static, 10 seconds):

  github.com/sudols/stock-order-book

  MIT Licensed · Open Source
  Bennett University — DTI Course 2026